Handin 3 Numerical Optimization

1 Introduction

This report derives equation (21) from the conjugate gradient framework using gradient orthogonality properties. We implement an approximate Newton method with conjugate gradients to solve the Newton system, analyzing convergence rates and computational efficiency across test functions in dimension $d = 20$.

2 Theory: Derivation of Equation (21)

Gradients in the Span of Conjugate Directions
From equation (20) with $v_{k+1} = -\nabla f(x_{k+1})$ we have

$$p_{k+1} = -\nabla f(x_{k+1}) - \sum_{i=0}^{k} \frac{\nabla f(x_{k+1})^T Q p_i}{p_i^T Q p_i} p_i.$$

Rearranging gives

$$\nabla f(x_{k+1}) = -p_{k+1} - \sum_{i=0}^{k} \frac{\nabla f(x_{k+1})^T Q p_i}{p_i^T Q p_i} p_i,$$

hence $\nabla f(x_{k+1}) \in \text{span}\{p_0, \ldots, p_{k+1}\}$. Since $p_0 = -\nabla f(x_0)$, an induction argument yields that for every k there exist coefficients $\gamma_0, \ldots, \gamma_k$ such that

$$\nabla f(x_k) = \sum_{i=0}^{k} \gamma_i p_i.$$

Orthogonality of Gradients
Let $j < k$. Using the representation above,

$$\nabla f(x_k)^T \nabla f(x_j) = \nabla f(x_k)^T \sum_{i=0}^{j} \gamma_i p_i = \sum_{i=0}^{j} \gamma_i \nabla f(x_k)^T p_i.$$

By Theorem 6, $\nabla f(x_k)^T p_i = 0$ for all $i < k$. Since $j < k$, all terms vanish and therefore

$$\nabla f(x_k)^T \nabla f(x_j) = 0.$$

Derivation of Equation (21)
We now derive equation (21). By induction, $p_i \in \text{span}\{\nabla f(x_0), \ldots, \nabla f(x_i)\}$, so we can write $Qp_i = \sum_{j=0}^{i} \alpha_j \nabla f(x_j)$ for suitable coefficients α_j. Therefore:

$$\nabla f(x_{k+1})^T Q p_i = \sum_{j=0}^{i} \alpha_j \nabla f(x_{k+1})^T \nabla f(x_j)$$

If $i < k$, then $j \leq i < k < k+1$, and by the gradient orthogonality shown above

$$\nabla f(x_{k+1})^T \nabla f(x_j) = 0 \implies \nabla f(x_{k+1})^T Q p_i = 0 \quad \text{for } i < k$$

In equation (20), all sums with $i < k$ vanish and only the term with $i = k$ remains. This gives:

$$p_{k+1} = -\nabla f(x_{k+1}) + \frac{\nabla f(x_{k+1})^T Q p_k}{p_k^T Q p_k} p_k$$

which is equation (21).

3 Programming: The Approximate Newton Algorithm

3.1 Implementation Details

We implement the approximate Newton method following the Inexact Newton algorithm (Algorithm 6). The Newton step is computed approximately by solving $\nabla^2 f(x_k)p_k = -\nabla f(x_k)$ with Conjugate Gradients using tolerance $\varepsilon_k = \eta_k \|\nabla f(x_k)\|$. This avoids explicit Hessian inversion and scales well for $d \geq 20$. A backtracking line-search satisfying the sufficient decrease condition ensures global convergence, and the stopping criterion is based on $\|\nabla f(x_k)\|$.

3.2 Selection of Test Functions

The inexact Newton framework assumes that the Hessian is positive definite for all x. To remain within this theoretical setting, we exclude f_2 and f_3. The Rosenbrock function f_2 is non-convex and exhibits saddle points for $d > 2$, while the transformation in f_3 may introduce negative eigenvalues, violating global positive definiteness. We therefore focus on f_1, f_4 and f_5. Function f_1 is a strictly convex baseline with diagonal Hessian, ensuring that the standard Newton direction $-H^{-1}\nabla f$ is well-defined and that quadratic convergence can be verified under ideal conditions. Function f_4 preserves global positive definiteness but introduces stronger anisotropy and non-trivial curvature, allowing us to study conditioning effects and impact on the number of CG iterations. Function f_5 is globally convex but has a singular Hessian at the optimum. This tests whether approximate Newton maintains stable convergence when curvature degenerates. This allow a meaningful comparison between Steepest Descent, exact and approximate Newton.

3.3 Performance Under Different Schedules

We evaluate the approximate Newton method under three schedules for η_k, designed to produce linear, superlinear and quadratic convergence. Figure 1 shows the decay of $|f(x_k) - f(x^*)|$ (log-scale) for f_1, f_4 and f_5.

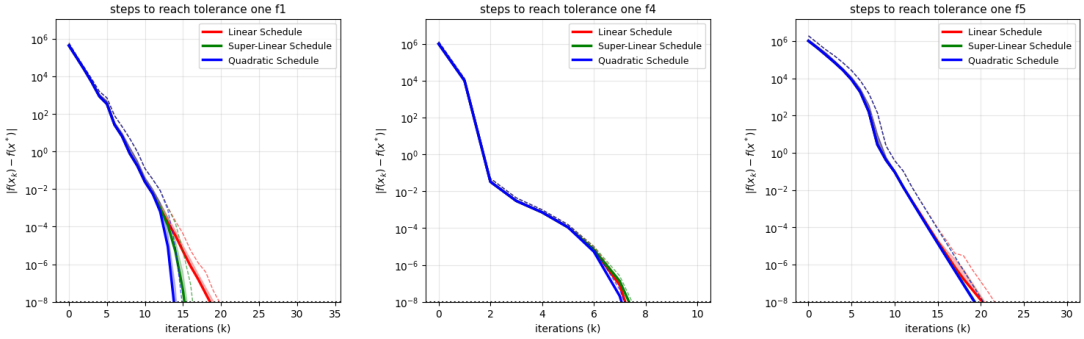


Figure 1: Eta schedule comparison $d = 20$, $\epsilon = 1e-8$,$\rho = 0.5$

We test a constant $\eta_k = 0.5$ (linear convergence), $\eta_k = \min\left(\frac{1}{2}, \|\nabla f(x_k)\|\right)$ (superlinear), and $\eta_k = \frac{1}{2}\min\left(\frac{1}{2}, \|\nabla f(x_k)\|\right)$ (superlinear, as used in Algorithm 6). We observe a difference in performance only on f_1. We attribute this to the fact that the theoretical rates derived in Lemma 4 are upper bounds on convergence, not exact descriptions.

$$\|\nabla f(x_{k+1})\| \leq \frac{L}{2}\left(\frac{1}{\lambda_1(x_k)} + \lambda_N(x_k)\eta_k\right)^2 \|\nabla f(x_k)\|^2 + \eta_k\|\nabla f(x_k)\|, \tag{1}$$

On well-conditioned problems all three schedules converge rapidly in practice regardless of the theoretical distinction, because the quadratic term already dominates and the actual convergence rate is much better than the bound suggests.

3.4 CG Steps During Optimization

With a quadratic convergence schedule, the number of CG steps per outer iteration increases sharply as $\|\nabla f(x_k)\| \to 0$, since the inner stopping criterion $\varepsilon_k = \eta_k \|\nabla f(x_k)\|$ becomes increasingly stringent. By Theorem 6, CG requires at most n steps to solve an n-dimensional system exactly, so the worst-case cost per Newton step grows linearly with dimension. This is observable in our experiments, where increasing the problem dimension leads to a rapid increase in total CG steps, underlining the computational advantage of using a looser schedule (linear or superlinear) in high-dimensional settings.

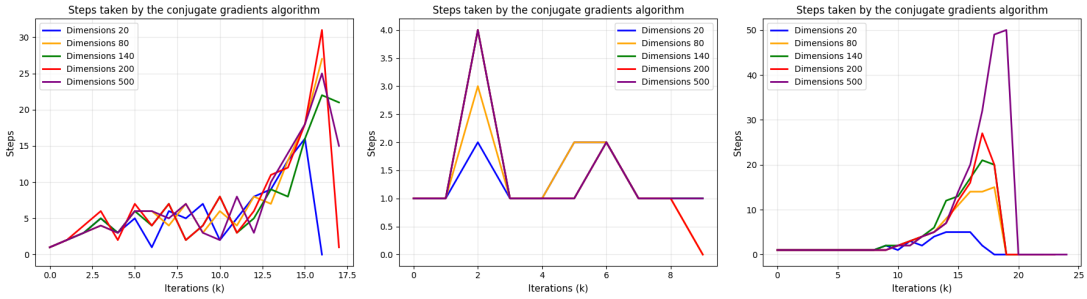


Figure 2: CG steps comparison $\epsilon = 1e-8$,$\rho = 0.5$, $\eta_k = (SuperLinear)$

3.5 Comparison of Methods

Figure 3 compares Steepest Descent, Newton-BT and Inexact Newton. For f_1 and f_4, both Newton variants converge in very few iterations, clearly outperforming Steepest Descent. The Inexact Newton method closely matches exact Newton, showing that approximate inner solves preserve fast local convergence. For f_5, convergence slows near the optimum due to Hessian degeneracy, reducing the advantage of Newton-type methods.

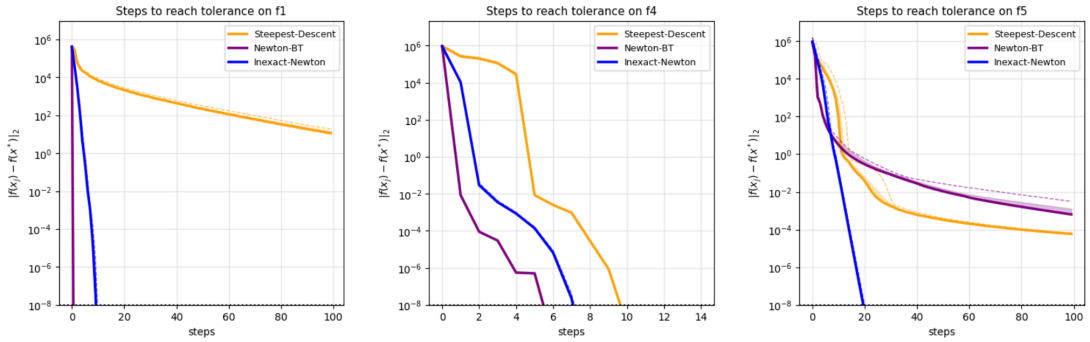


Figure 3: Newton's comparison $d = 20$, $\epsilon = 1e-8$,$\rho = 0.5$, $\eta_k = (SuperLinear)$

4 Conclusion

Deriving equation (21) shows how gradient orthogonality makes the CG recurrence simpler. The approximate Newton method, solved via CG, achieves comparable performance to exact Newton while avoiding Hessian inversion. On well-conditioned functions (f_1, f_4), the method converges in few iterations regardless of the η_k schedule, demonstrating the robustness of inexact Newton methods. The number of CG steps depends critically on function structure and proximity to the optimum, underscoring that computational efficiency is function-dependent rather than dimension-dependent alone.